Learning from Earlier Websites: Cross-Task Long-Term Memory for a Small Website-Generation Model



Qiaopan Ma

Supervisor:
Qian Liu

School of Computer Science
The University of Auckland

A dissertation submitted in partial fulfilment of the requirements for the degree of
Master in Artificial Intelligence,
the University of Auckland, 2026.

This dissertation is for examination purposes only and is confidential to the examination process.

Abstract

Website-generation agents solve every task from scratch, forgetting what went wrong on earlier websites. This dissertation asks whether a system built around a *small* generator, WebGen-LM-7B, can improve across a sequence of WebGen-Bench tasks when it is given a long-term memory built from its own earlier attempts, without changing its weights and while retaining the official single-attempt generation and evaluation of each task. A separate, larger model maintains the memory.

We reproduce the official No Memory baseline on a self-hosted cluster (28.5% accuracy on 101 tasks; 28.4% reported). Its failures show that many zero-score tasks are blank pages caused by basic code errors, such as importing a package that is not installed, and, in a nine-task diagnostic, short textual warnings did not reliably prevent these failures. A text-dominated memory improved early performance but later fell behind No Memory; the experiments do not isolate the cause of this decline.

The final method, code memory, adds a requirement checklist and a code pool: when earlier tasks show a code error that breaks a site, the prompt carries a short fragment copied verbatim from the chain's own compiled, successful websites. In a single run over all 101 tasks it reaches 35.2% against 28.5% (paired gain 7.1 points, 95% CI 0.9–13.1) (+7.9 points averaged over five task orders), and it gains 7.4 and 5.5 points in two shuffled orders, one of which was used to develop it. An intermediate version that sometimes showed whole files scored higher in the original order (40.0%) but made the generator return the untouched starter project in shuffled orders; fragments removed this failure. Gains are functional, largely from sites that now render; appearance on sites working in both arms is similar.

In these runs, short code fragments reduced the targeted failures, whereas textual warnings did not reliably prevent them. All results are single runs on the benchmark on which the method was developed. [TODO: Status: fixed-prefix ablation queued; its result decides whether the gain is described as accumulated memory or as automatically discovered prompt code.]

Acknowledgements

I thank my supervisor, Dr Qian Liu, for supervising this project and for the freedom and support to pursue it, and Ziqin Zhu, a PhD student in his group, for generous guidance on framing research questions, designing fair experiments and structuring a dissertation. I thank Tanya Carter for coordinating the research project course, and the School of Computer Science for access to the CS ML GPU cluster, on whose H200 GPUs all experiments were run. Finally, I thank my family for their support.

Qiaopan Ma
October, 2026

Table of Contents

List of Figures **v**

List of Tables **vi**

1 Introduction **1**

1.1 Background 1
1.2 Problem Statement 2
1.3 Research Questions 3
1.4 Scope 3
1.5 Contributions 4
1.6 Organisation of Thesis 4

2 Background and Related Work **5**

2.1 Website Generation Benchmarks 5
2.2 Memory for LLM Agents 6
2.3 Small Models, Long Prompts and Instruction Following 7
2.4 Self-Correction and Debugging 7
2.5 Research Gap 7

3 Experimental Setup **9**

3.1 Benchmark and Protocol 9
3.2 The Baseline: How One Task Is Generated and Scored 10
3.3 Models and Roles 11
3.4 Metrics 12
3.5 Experimental Design 13
3.6 Reproducing the Baseline 13
3.7 Reproducibility and Constraints 14

4 Method **16**
4.1 Problem Formulation 16
4.2 Starting Point: Why No Memory Fails 17
4.3 Final Method 18

5 Results **22**
5.1 Main Comparison 22
5.2 Learning Curve 23
5.3 Why Text Memory Declines and Code Memory Holds 23
5.4 Error Analysis 25
5.5 Ablations: Checklist Only and Fixed Code Prefix 25
5.6 Robustness to Task Order 26

6 Discussion **31**
6.1 The Role of Memory Representation 31
6.2 What Is Learned and at What Cost 31
6.3 Limitations and Threats to Validity 32
6.4 Ethical Considerations and Impact 34
6.5 Project Management 34

7 Conclusion **36**
7.1 Answers to the Research Questions 36
7.2 Future Work 37

Bibliography **38**

List of Figures

3.1 One task in the pipeline. Black: the official WebGen-Bench setup (No Memory). Blue: the memory module, which reflects after each task and adds text to the prompt of later tasks only. 9

3.2 The first screenshot the tester received for task 000006 (seed-11 order). (a) An undeclared package stops the whole site. (b) The numbered dashed boxes are drawn by the tester to name the elements it can click; they are not part of the website. 11

3.3 The three memory designs. Each arrow names the failure that motivated the next design. Accuracy is on the manifest order (101 tasks, one run). . 13

4.1 The memory flow of the final method. 17

4.2 The final method's prompt for task 000041 (seed 11), shortened: requirement, checklist with attached lessons, and verified code fragments. 19

5.1 Cumulative and rolling-10 accuracy by execution order for No Memory, text memory and both code memories. 24

5.2 Rolling 10-task accuracy gap to No Memory in each order (positive: memory better). Blue: final method; orange: whole-file code memory (manifest, seed 11 and seed 29 only). Seed 47 and interleaved are compared with the per-task mean of three No Memory runs. The orange dip after task 45 in seed 11 coincides with the scaffold outputs (Figure 5.3). 28

5.3 Task 000041 (seed 11), whole-file code memory. Left: the injected `App.vue` of an earlier food-donation site. Right: the generator returned the untouched starter project; all five test cases failed. 29

List of Tables

2.1 Closest related studies. Update: how the memory changes across tasks. Consumer: the model that uses the memory. Order: results compared across task orders. “no” and “–” mean not evaluated, not a shown limitation. 8

3.1 Model roles. The appearance grader is the only deviation from the paper, which uses GPT-4o. 11

3.2 Experimental design. Memory content is what is added to the generator’s prompt; the designs are those of Figure 3.3. 14

4.1 Constants of the final method not stated in the text. Some were revised after development runs (mainly seed 11, Section 5.6.1); all were fixed before the final runs and none was tuned systematically. 21

5.1 Main results on all 101 tasks in the manifest order (single run per arm). Yes, Partial, No and Start failed are percentages of the 647 test cases; Accuracy = Yes $+0.5\times$ Partial. Appearance is the mean grade over all 101 tasks (1–5; tasks without a working site count as 0). Paired Δ is the mean per-task difference to No Memory with a 95% bootstrap confidence interval (CI); it can differ from the difference of pooled accuracies because tasks have different numbers of test cases. Zero: tasks where every test case fails. 23

5.2 Prompt content by segment (tasks 1–30 / 31–60 / 61–101). 24

5.3 Generated sites with code problems (number of tasks; 1–30 / 31–60 / 61–101, total in brackets). 24

5.4 Ablation over the 101 tasks in the manifest order (single runs). Accuracy is pooled over checks; appearance averages tasks with a grade. 26

5.5 Task-order experiments: accuracy (%) of No Memory (NM), whole-file code memory (WF) and the final method with fragments (F), paired differences with 95% bootstrap CI over tasks, and zero-score tasks. Single run of each arm per order. 27

Chapter 1

Introduction

1.1 Background

1.1.1 Website Generation with Language Models

Large Language Model (LLM) agents can translate a natural-language requirement into a runnable website [19]. WebGen-Bench evaluates this capability end to end. It starts the generated project and uses a Vision-Language Model (VLM) tester to check requested features through the interface [19, 11]. A website that appears plausible but does not support the required interactions therefore does not pass.

The strongest benchmark results use large proprietary models. Smaller open models are attractive for a different set of reasons: they can run locally, cost less to operate, and do not require task data to be sent to a third party. WebGen-LM-7B, the 7-billion-parameter generator released with WebGen-Bench, was fine-tuned for this setting but reaches 28.4% accuracy under the official protocol [19]. Improving such a model without retraining is therefore a practical problem as well as a research question.

1.1.2 Every Task Is Solved in Isolation

WebGen-Bench, like most code-generation benchmarks, treats each task independently. The agent begins every task with the same weights and an empty context. If it produces a blank page because it imports a package that is not installed, the same failure can recur on the next task. In our baseline reproduction (Section 4.2), almost one third of the 101 tasks received zero. Many shared a small number of causes: an undeclared dependency, a missing import, or a template that did not compile.

A developer would normally avoid a build error after encountering it once. This study asks whether an agent can retain and use that kind of experience across tasks.

1.1.3 Long-Term Memory for Agents

Agent-memory systems aim to support this form of reuse. After a task, an agent may reflect on what occurred, store a lesson or procedure, and retrieve it when a related task appears later [36, 31, 21, 32, 5]. Most such systems use large models and store memory as natural-language insight. Small models are a harder case. They follow multi-instruction prompts less reliably [10, 13, 17], are distracted by irrelevant context [24], and lose accuracy as prompts grow [8, 18]. Every stored lesson lengthens the prompt, so a memory that helps a large model can bury the requirement for a 7B generator or give it advice it cannot turn into code. Which *form* of memory such a generator can use remains open.

1.2 Problem Statement

This dissertation examines whether a system built around a small website generator can improve through experience without changing the generator's weights. Here, *long-term memory* means information written after one task, retained across subsequent tasks, retrieved under an explicit condition, and added to later prompts. *Learning* therefore refers to an outcome-dependent change in the system's later behaviour that is carried by external memory. The generator parameters remain fixed, while a separate, larger memory model maintains the memory (Chapter 3).

The study uses WebGen-Bench with WebGen-LM-7B. The generator, Bolt.diy scaffold, and official evaluation protocol are held constant. The treatment changes only the generator prompt through an external memory pipeline, which also adds computation (Chapter 6). Memory is derived solely from earlier tasks completed by the agent and their evaluation outcomes. The hidden evaluation for the current task is never used, preserving a fair comparison with the no-memory baseline.

This setting makes cross-task memory difficult to assess for two reasons. First, tasks arrive sequentially, so the memory available for a task depends on the tasks that precede it; results may therefore vary with task order. Second, the official protocol permits only one attempt per task. The memory must help the model avoid an error before it occurs rather than repair it afterwards.

1.3 Research Questions

RQ1. Can cross-task long-term memory, built only from an agent's earlier tasks, improve the functional accuracy of a small website generator under the official WebGen-Bench protocol?

RQ2. Which form of memory can a small generator use effectively: textual lessons, structured requirement checklists, or code examples?

RQ3. How does any benefit change as experience accumulates, and how sensitive is it to the order in which tasks are presented?

RQ3 separates learning from experience from a fixed prompt change that would help equally at every point in a run. The fixed-prefix ablation (Section 5.5) is designed to test this distinction.

1.4 Scope

The study is intentionally narrow so that the comparisons remain interpretable.

- **One benchmark, generator, and scaffold.** All experiments use the 101 WebGen-Bench tasks, WebGen-LM-7B, and Bolt.diy. Larger generators and alternative agent scaffolds, such as OpenHands and Aider, are outside the study because changing the agent would also change how memory reaches the model.
- **Official protocol without repair.** Each website is generated once and evaluated as produced, following the original benchmark. Build checks and repair loops are excluded because they would make the comparison with the published baseline less direct.
- **No retraining.** Memory changes the prompt only. Fine-tuning the generator on stored experience is beyond the scope of this work.
- **Self-hosted models only.** Generation, evaluation, and memory processing run on a university GPU cluster; no commercial API is used.

1.5 Contributions

1. A fully self-hosted reproduction of the WebGen-Bench no-memory baseline for WebGen-LM-7B (28.5% versus the reported 28.4%), together with a failure analysis showing that many zero scores arise from blank pages caused by basic code errors (Chapter 3 and Section 4.2).

2. A long-term memory pipeline for website generation consisting of a requirement checklist, a code pool containing fragments copied verbatim from the chain's own compiled websites, and memory-hygiene rules. Each component was introduced in response to an observed failure in an earlier version (Chapter 4).

3. Evidence that, in these runs, short code fragments reduced targeted failures for this small generator where textual lessons did not, together with an ablation that separates the checklist from stored cross-task content (Chapter 5).

4. A task-order study using alternating paired runs. It shows that a memory entry can become harmful under a different ordering and that replacing whole-file examples with fragments removes that specific failure mode (Sections 5.6 and 5.6.1).

1.6 Organisation of Thesis

Chapter 2 reviews related work and identifies the research gap. Chapter 3 describes the benchmark, models, and reproduced baseline. Chapter 4 presents the memory method and the failures that led to it. Chapter 5 reports the results. Chapter 6 discusses their interpretation and limitations, and Chapter 7 answers the research questions and outlines future work.

Chapter 2

Background and Related Work

2.1 Website Generation Benchmarks

A website consists of multiple files, build configuration, and interaction logic. A project may look complete while failing to start or rendering only a blank page. Recent benchmarks therefore execute generated projects and test them through their interfaces. Related benchmarks cover interaction with existing websites (Mind2Web and WebArena [6, 40]), generation from screenshots (Design2Code [27]), and repository repair (SWE-bench [14]).

WebGen-Bench [19] contains 101 website requirements across 20 application domains and 647 functional test cases. For each task, the generated project is installed and started, then a Graphical User Interface (GUI) agent based on WebVoyager [11] executes the test cases and returns *Pass*, *Partial*, or *Fail*. A separate VLM assigns a landing-page appearance score from 1 to 5. The benchmark also introduces WebGen-LM, a set of generators fine-tuned from Qwen2.5-Coder [12] on agent trajectories. With the Bolt.diy scaffold, WebGen-LM-7B achieves 28.4% accuracy and an appearance score of 2.5. Larger general models, including Qwen2.5-Coder-32B, score lower (9.5%), leaving the 7B model sufficiently capable for improvements to be observable without being close to the benchmark ceiling.

WebGen-Bench evaluates a generator within an agent scaffold rather than in isolation. In the Bolt.diy scaffold used here, the model writes a project through a sequence of file actions. The scaffold applies those actions to a template project, installs dependencies, and starts a development server. The official protocol gives the generator one attempt: it does not run a build check or a repair loop, and a project that cannot start receives zero on all its functional test cases. Consequently, memory can affect only what

the generator writes on its first attempt; it cannot repair an error after the benchmark has exposed it.

The benchmark reports single runs without error bars because repeated runs are costly [19]. It also defines no interaction between tasks. Once memory is introduced, task order becomes a treatment variable, which Section 5.6 addresses explicitly.

2.2 Memory for LLM Agents

An agent-memory system must decide what to store after a task, how to maintain that information, and how to retrieve it for a new task.

Text and code memory. ExpeL [36] extracts natural-language insights from trajectories and maintains them through editing and voting. ReasoningBank [21] distils strategies from successes and failures. Generative Agents [23] rank memories by recency, importance, and relevance. Reflexion [25] reuses self-reflections when retrying the same task. Voyager [31] and Agent Workflow Memory [32] store skills or workflows. Metis [5] promotes recurring plans to executable tools; with a GPT-4o executor these tools reach 53.3% task completion, against 73.3% for text memory. Synapse [38], Dynamic Cheatsheet [29], and ACE [35] keep abstracted exemplars or an incrementally edited playbook. From this work, the present study adopts two maintenance principles: remove source-task details when possible, and version entries rather than overwriting them. Unlike Metis, the present method supplies code fragments to the generator as prompt content; it does not execute them as tools.

Transfer and cross-task reuse. Memory Transfer Learning [15] reports that abstract insights transfer more effectively than concrete trajectories when strong models are used, while concrete trajectories can cause negative transfer. Neither that work nor Metis evaluates small models. Super Library Agent [28] generates WebGen-Bench applications sequentially with a shared component library. It focuses on maintainability, uses a strong backbone, evaluates 24 single-topic tasks, and reports no clear functional-accuracy improvement (76.04% to 77.21%). It neither stores lessons from failures nor varies task order.

Maintenance and task order. Mem0 [4] chooses whether a new memory should be added, update an existing entry, or be skipped. Memento [39] estimates case utility

from outcomes, while MemGPT [22] manages memory tiers. This study borrows write-time deduplication and a utility score. Evo-Memory [33] and AgentCL [26] show that the composition and ordering of a task stream can affect results. Section 5.6 therefore compares orders with adjacent same-domain tasks and orders in which domains are mixed.

2.3 Small Models, Long Prompts and Instruction Following

Adding information to a prompt has a cost. As the number of instructions rises, models become less likely to satisfy all of them [10, 13]. Information in the middle of a long context is used less than information near either end [18]. Irrelevant context distracts models [24], and longer inputs alone can reduce performance even when retrieval is accurate [8]. Models can be trained to prioritise instructions by source [30], which motivates keeping the user requirement above any memory content. Small models also gain less consistently from long reasoning chains [17].

These findings come mostly from reasoning and question answering, not code generation. For a memory system the consequence is direct: each injected lesson is one more instruction placed after the requirement, so a growing store lengthens every prompt and can push the requirement towards the less-attended middle or add advice the generator cannot convert into code.

2.4 Self-Correction and Debugging

Reflection on failed attempts is related to Self-Refine [20], Self-Debug [3], and Chain-of-Verification [7]. Those methods repair the *same* task using feedback. The formal comparison here does not allow that behaviour, because using hidden evaluation results to repair a benchmark task would leak the answer. Reflection is used only to write memory for *later* tasks. The reflection model follows investigation steps adapted from systematic debugging [34].

2.5 Research Gap

Table 2.1 compares the closest studies along the dimensions relevant here. Cross-task memory has almost exclusively been studied with large agents, which leaves open

whether a small generator benefits and which representation it can use. On WebGen-Bench itself, the only cross-task study addresses maintainability with a strong backbone on single-topic suites and finds no clear functional-accuracy gain. Memory studies also usually report a single task order, even though the memory available for a task depends on the tasks before it.

This dissertation addresses these gaps by measuring memory effects on functional accuracy for a 7B generator while retaining the official single-attempt procedure within each task; by examining textual, checklist, and code-based memory across system versions, while recognising that these versions differ in more than representation; and by repeating comparisons under different task orders with task-paired confidence intervals.

Table 2.1: Closest related studies. Update: how the memory changes across tasks. Consumer: the model that uses the memory. Order: results compared across task orders. "no" and "–" mean not evaluated, not a shown limitation.

Study	Memory form	Update	Consumer	Website tasks	Order
ExpeL [36]	textual insights	add, edit, up/down-vote	large	–	no
ReasoningBank [21]	reasoning strategies	append from successes and failures	large	–	no
Voyager [31]	executable skills	add verified skills	large	–	no
Metis [5]	text + executed code tools	promote recurring procedures	large	–	no
Memory Transfer Learning [15]	four representations	fixed (transfer study)	large	–	no
Super Library Agent [28]	shared component library	library grows with tasks	large	24 tasks	no
Evo-Memory [33]	benchmark of memory methods	varies	various	–	yes
This work	checklist + code fragments + text	dedup, utility retirement, versioned code	**7B**	**101 tasks**	**yes**

Chapter 3

Experimental Setup

3.1 Benchmark and Protocol

All 101 WebGen-Bench tasks are used. They cover 20 application domains, with two to eight tasks per domain. The benchmark manifest groups tasks into domain blocks; Section 5.6 additionally evaluates shuffled orders. The experiments follow the official protocol. The generator produces each project once, no repair loop is used, and a project is evaluated once its dependencies install and its development server starts. Test cases without an evaluation record count as failed, as in the official scoring script. An earlier project phase used a stricter build gate (`vue-tsc`) and a more permissive repair loop. Those results are not comparable with the published protocol and are not reported as main results.

Figure 3.1 places the memory module in the evaluation pipeline. It is the sole component that differs across arms. The generator, scaffold, tester, and scoring procedure are shared.

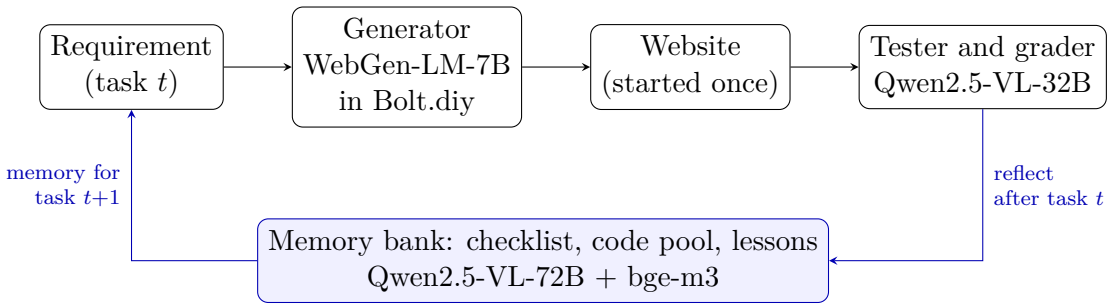


Figure 3.1: One task in the pipeline. Black: the official WebGen-Bench setup (No Memory). Blue: the memory module, which reflects after each task and adds text to the prompt of later tasks only.

3.2 The Baseline: How One Task Is Generated and Scored

The no-memory baseline, called *No Memory*, is the unmodified WebGen-Bench pipeline [19]. It is used because it matches the setting reported in the benchmark paper, allowing the reproduced result to be compared with the published 28.4% accuracy (28.5% here, Section 3.6), and because it leaves memory as the only difference between experimental arms. A single task, 000006, proceeds as follows:

```
Please implement a solar dashboard to display real-time and historical solar
data. [...] Configure the background color to honeydew, with components using
dark olive green.
```

1. **Prompt.** The requirement above is the complete prompt. No additional text is supplied.

2. **Generation.** Bolt.diy, an open-source browser-based coding agent, opens a Vite + Vue + TypeScript starter project whose `package.json` declares only `vue`. The generator replies once. Bolt.diy writes the files in that reply, installs dependencies, and starts the development server. There is no build check, error feedback, or second attempt.

3. **Functional testing.** The benchmark provides hidden test cases, averaging roughly six per task (647 in total); for this task, for example, "Verify the display of real-time solar power generation data", with an expected result. A WebVoyager-style GUI agent [11] receives one case at a time, sees a screenshot with numbered clickable elements, and may click or type for up to 15 steps. It then returns YES (the expected result is met), PARTIAL (met in part), or NO. If the site does not start, every test case is recorded as *start failed*.

4. **Appearance grading.** A vision model receives a full-page landing-page screenshot and the requirement, then assigns a score from 1 to 5 for rendering, content relevance, layout harmony, and modernness.

The tester and grader are model-based judges and can introduce their own biases [37], so their raw transcripts are retained. Memory affects only the prompt in step 1, where it adds text after the requirement (Chapter 4); generation and both evaluation stages remain unchanged across arms.

Figure 3.2 shows what the tester saw for this task in two runs. Under No Memory the model imported the chart library `vue-chartjs`, which the template does not declare; Vite cannot resolve the import, the page shows only the error overlay, and all test cases fail. With the final memory the generator built the three requested views without the undeclared package, and all test cases passed.



(a) No Memory: score 0.

(b) Code memory (fragments): score 1.0.

Figure 3.2: The first screenshot the tester received for task 000006 (seed-11 order). (a) An undeclared package stops the whole site. (b) The numbered dashed boxes are drawn by the tester to name the elements it can click; they are not part of the website.

3.3 Models and Roles

All models are self-hosted on the university CS ML cluster. Each job uses two NVIDIA H200 GPUs and serves models through vLLM [16]. No commercial LLM service is used. Table 3.1 lists the model assigned to each role.

Table 3.1: Model roles. The appearance grader is the only deviation from the paper, which uses GPT-4o.

Role	Model
Generator	WebGen-LM-7B (BF16), Bolt.diy scaffold, single generation
Functional tester	Qwen2.5-VL-32B-Instruct [1] (WebVoyager-style GUI agent)
Appearance grader	Qwen2.5-VL-32B-Instruct (paper: GPT-4o)
Memory / reflection	Qwen2.5-VL-72B-Instruct-AWQ
Embeddings	bge-m3 [2]

The generator. WebGen-LM-7B, released with WebGen-Bench [19], is Qwen2.5-Coder-7B-Instruct [12] fine-tuned on Bolt.diy trajectories for instructions decontaminated against the benchmark. It already knows the Bolt.diy output format, which makes it the natural small baseline: a model distilled for exactly this scaffold.

The memory model differs from the evaluator so that reflection cannot imitate the evaluator's grading behaviour. vLLM otherwise applies the generation configuration distributed with each model repository. WebGen-LM-7B includes a repetition penalty that Bolt.diy does not send, so it is disabled for the generator to reproduce the paper's setting.

3.4 Metrics

The benchmark metrics are retained so that the reproduced baseline can be compared directly with the published result. For each arm, *Yes*, *Partial*, *No* and *Start failed* are the percentages of the 647 cases with each outcome, and

$$\text{accuracy} = \text{Yes} + 0.5 \times \text{Partial}, \tag{3.1}$$

so a partial result counts as 0.5, while a site that does not start contributes zero on every test case. This is the official accuracy, pooled over test cases. The same calculation gives *task accuracy* $(p + 0.5\,q)/n$, where p cases pass, q are partial, and n is the number of cases for that task. A *zero-score* task fails every case, commonly because the page is blank. *Appearance* is the mean 1–5 grade; in the main table, tasks without a working site receive 0.

Test cases within a task are not independent because they share one generated website and often fail together when it does not start. Arms are therefore compared task by task. The reported paired effect is the mean difference in task accuracy, with a 95% percentile bootstrap confidence interval (CI) produced by resampling the 101 tasks [9], alongside counts of wins, losses, and ties. This interval describes variation over the tasks in the benchmark rather than variation across repeated memory chains, which is approximately five points (Section 5.6.1). The text therefore states whether an interval excludes zero rather than describing a result as statistically significant. Learning curves average task accuracy.

3.5 Experimental Design

Three memory designs are compared (Figure 3.3; Chapter 4). *Text memory* stores natural-language lessons and attaches them to a requirement checklist. *Code memory (whole-file)* retains the checklist and supplements it with examples from the chain's own working websites, some of which are entire files. *Code memory (fragments)*, the final method, presents only short verified fragments. It is treated as the final method because it removed the scaffold failure observed under shuffled orders, not because it scores highest in every order: whole-file code memory scores higher in manifest order (Section 5.1). The method was refined over several iterations, but only these three designs were run across all 101 tasks and are reported here.[1]

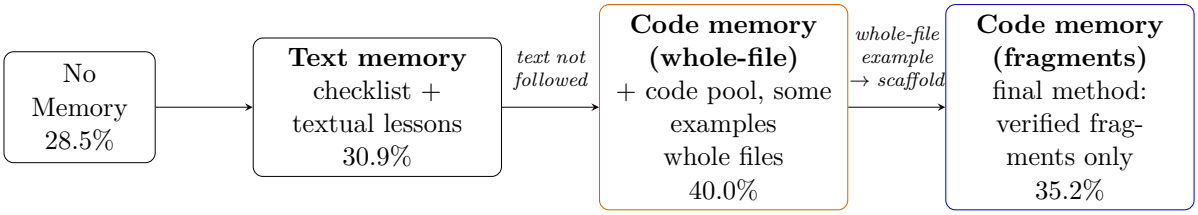


Figure 3.3: The three memory designs. Each arrow names the failure that motivated the next design. Accuracy is on the manifest order (101 tasks, one run).

Table 3.2 identifies each arm, the orders in which it was run, and its purpose. Every arm uses WebGen-LM-7B with Bolt.diy under the official protocol, covers all 101 tasks, and begins with empty memory. Each arm is run once for each order. The order definitions appear in Section 5.6.

Memory never accesses information from the current task (leave-one-task-out), and its evaluation outcome is never fed back into that task.

3.6 Reproducing the Baseline

Across all 101 tasks, No Memory achieves 28.5% accuracy, compared with 28.4% in the paper. Its mean appearance score is 2.35, versus 2.5 in the paper, which used GPT-4o for appearance grading, and it has 30 zero-score tasks. The close aggregate-accuracy match supports an approximate reproduction of the published self-hosted pipeline. The appearance graders differ, and task-level agreement cannot be assessed because the paper does not release per-task outputs.

[1] In the code repository these runs are labelled v4, v7 and v9.1.

Table 3.2: Experimental design. Memory content is what is added to the generator's prompt; the designs are those of Figure 3.3.

Arm	Memory content	Task orders	Question
No Memory	none (requirement only)	manifest, seeds 11, 29, 47, interleaved	Does the baseline reproduce the paper, and how much does it vary between runs?
Text memory	checklist with textual lessons	manifest	Does textual memory help a small generator?
Checklist only	checklist; memory neither read nor written	manifest	How much does decomposing the requirement add?
Code memory (whole-file)	checklist, lessons and code examples, some complete files	manifest, seeds 11, 29	Does code memory help, and does it survive a change of order?
Fixed code prefix	checklist and the final code memories from task 1; no memory	manifest	Is the gain learning, or just code in the prompt?
Code memory (fragments)	as above, but code examples are verified fragments only	manifest, seeds 11, 29, 47, interleaved	Does the final method hold across task orders?

3.7 Reproducibility and Constraints

Each run begins with empty memory. The project archive retains its configuration file, task order, prompts, retrieved and injected memory, raw evaluator transcripts, and scores. Generator decoding is deterministic, so prompt differences explain differences between arms, although the GUI tester itself is not deterministic.

Hardware. One long-running Slurm job shared two NVIDIA H200 GPUs across all models and arms through vLLM. All arms share the same allocation: the generator and the 32B tester on one GPU, the 72B memory model and the embedding model on the other. Per task, No Memory takes about 6 minutes and Long Memory about 12 minutes of wall time (medians measured in the seed-11 and seed-29 runs, where the two arms alternate on a single queue so that only one task runs at a time: 5.7 and 5.5 minutes for No Memory, 11.3–11.9 minutes for Long Memory), the difference coming mainly from the memory model's checklist, selection and reflection calls.

Code and data availability. The code, configuration files, and archived run records, including prompts, injected memory, scores, and memory databases, are publicly available at https://github.com/qiaopan/WebGen-Bench; the run records are in `archive/phase2-webgen-2026-10/`.

Compute is the main practical constraint. A memory task takes approximately 12 minutes, compared with roughly 6 minutes for No Memory, so a 101-task run lasts about a day. As in the original benchmark, each configuration is run once per order. Comparisons are paired by task, and No Memory is repeated across orders to estimate run-to-run variation.

Chapter 4

Method

4.1 Problem Formulation

A run processes tasks $t_1, \ldots, t_n$ ($n = 101$) in a fixed order. Task t_i has a requirement r_i and hidden test cases $c_{i,1}, \ldots, c_{i,m_i}$. A generator G with fixed parameters maps prompt x_i to project $y_i = G(x_i)$, and evaluator E assigns accuracy $a_i = E(y_i, c_{i,\cdot}) \in [0, 1]$ as defined in Chapter 3. With No Memory, $x_i = r_i$. With memory,

$$x_i = r_i \oplus f(r_i, M_{i-1}), \qquad M_i = u\big(M_{i-1}, r_i, y_i, \tau_i, a_i\big), \tag{4.1}$$

where $M_0 = \emptyset$, f selects and formats information from the memory available before task t_i, $\oplus$ appends it to the requirement, τ_i is the evaluator transcript, and u updates the memory only after the task has been evaluated. Because f reads only M_{i-1}, evaluation of t_i cannot affect its own prompt. The generator, evaluator, and protocol are identical across arms. Algorithm 1 and Figure 4.1 describe the final design (Section 4.3); Section 4.2 first presents the failures that motivated it.

Algorithm 1 Cross-task memory for one run (final version).

$S_0 = (M_0, F_0, C_0) \leftarrow (\emptyset, \emptyset, \emptyset)$ ▷ lessons, code facts, versioned code pool
for $i = 1, \ldots, n$ **do**
  $L \leftarrow \textsc{Checklist}(r_i)$ with at most three hard-match lessons from M_{i-1}
  $P \leftarrow \textsc{SelectActive}(C_{i-1}, r_i)$ ▷ ≤ 3 active entries
  $x_i \leftarrow r_i \oplus L \oplus P$; $y_i \leftarrow G(x_i)$; $(a_i, \tau_i) \leftarrow E(y_i)$
  $M_i \leftarrow \textsc{Reflect}(M_{i-1}, r_i, y_i, \tau_i, a_i)$ ▷ deduplicate; update utilities
  $F_i \leftarrow F_{i-1} \cup \textsc{StaticAnalyse}(y_i)$
  $C_i \leftarrow \textsc{UpdatePool}(C_{i-1}, F_i, \{y_j : j \leq i,\ a_j \geq 0.5\})$ ▷ activate, verify, version
end for

StaticAnalyse records undeclared imports, Vue functions used without import, and compile errors. UpdatePool is described in Section 4.3.2, while Table 4.1 gives the fixed constants.

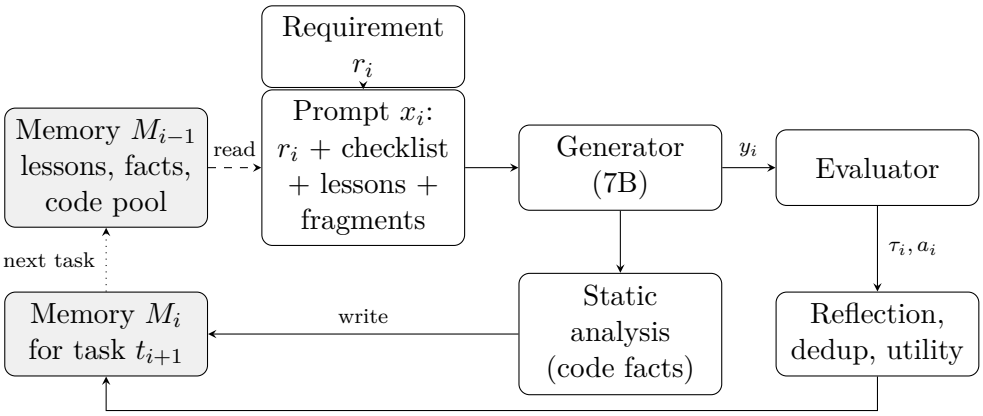


Figure 4.1: The memory flow of the final method. Dashed arrows read the memory built from earlier tasks; the lower row writes memory after evaluation. r_i: requirement; y_i: generated site; τ_i: evaluator transcript; a_i: score; M_{i-1}, M_i: memory before and after t_i.

4.2 Starting Point: Why No Memory Fails

The initial analysis examined generated code rather than scores alone. Two observations shaped the later design.

Blank pages from basic errors. Bolt.diy's Vue template declares only `vue` as a dependency, and WebGen-LM-7B did not edit `package.json` in any of the 81 inspected responses. When it imported `vue-router`, or used a Vue function such as `ref` without importing it, the site rendered as a blank page and all test cases failed. In an early Long Memory run, 26 of 74 tasks produced blank pages; 19 of those failures involved an undeclared `vue-router` import. In many cases, functional difficulty cannot be assessed because the site fails before any requested behaviour can be evaluated.

Text is not followed. A targeted diagnostic, rather than a powered experiment, used nine tasks that had produced blank pages. Their original prompts were retained while one sentence was added in several forms: "add vue-router to package.json", "do not use vue-router", "put pages in App.vue with `ref` and `v-if`", or the same instruction at both the beginning and end of the prompt. No phrasing recovered more than one task out of three; each was tried on three of the nine tasks, and a task counted as recovered

when the generated code imported no undeclared package and compiled. A ten-line code example implementing the same idea, tested on all nine tasks, recovered seven. The text variants and code example were evaluated on different task subsets, so this finding is suggestive rather than conclusive. It nevertheless indicated that code might be a more usable memory representation for this model, consistent with evidence that small models follow instructions less reliably [10, 17].

4.3 Final Method

For every task, the pipeline proceeds in the following order.

1. **Build a requirement checklist.** The memory model divides the requirement into items (Section 4.3.1).
2. **Retrieve lessons.** It also decomposes the requirement into no more than three functional *aspects*, each defined by a feature, user trigger, expected state change, and domain-independent interaction pattern, plus appearance aspects. Functional lessons are retrieved by similarity and assessed as a hard match (the same trigger and state change), soft transfer (the same interaction type), or reject. Only hard matches are attached to checklist items. A small common pool contributes site-wide lessons containing a concrete code construct.
3. **Select code.** At most three verified fragments are added when their trigger conditions hold (Section 4.3.2).
4. **Generate and evaluate.** WebGen-LM-7B generates one project, which then undergoes the official functional and appearance evaluation.
5. **Reflect and store.** The memory model reflects on the evaluated task. New lessons are deduplicated, utility estimates are updated, and code facts are recorded for the pool.

The products of step 5 can influence only later tasks through steps 2 and 3. Figure 4.2 illustrates the resulting prompt. Under No Memory, the first requirement paragraph is the full prompt.

4.3.1 Requirement Checklist

The original requirement remains at the beginning of the prompt and is explicitly binding. The checklist repeats each requested feature so that added memory does not displace

```
Please implement an online trading website for bidding and trading. [...] Set the
background color to lemon chiffon and the component color to chocolate.

FEATURE CHECKLIST (the requirement above, item by item; build every item):
1. Please implement an online trading website for bidding and trading.
3. allowing users to browse trading opportunities,
   Note from earlier websites: For the trading opportunities: use `v-for` to
   display the list with current status.
[... items 2, 4-8 ...]

CODE FROM EARLIER WEBSITES THAT WORKED:
Component fails: ref() not imported. Earlier websites broke here (a component
calls ref() without importing it from 'vue' (that page fails when opened),
19 tasks). Code that works:
    import { ref, onMounted } from 'vue'
    const wheelRef = ref<HTMLElement | null>(null)
Vue compile error: v-model on prop. [...] (4 tasks). Code that works:
    <SearchBar :modelValue="searchQuery"
               @update:modelValue="emit('update:searchQuery', $event)" />
[... one more entry ...]

Build every checklist item; the notes only warn about mistakes earlier websites made.
```

Figure 4.2: The final method's prompt for task 000041 (seed 11), shortened: requirement, checklist with attached lessons, and verified code fragments.

a requirement from the model's attention [18, 30]. Lessons are attached to the checklist item to which they apply, rather than presented as a separate list that could be mistaken for the task's complete scope.

4.3.2 Code Pool

Detection for the code pool does not rely on an LLM.

- **Problem facts.** After each task, static analysis records undeclared imports (`import:<package>`), Vue functions used without import (`vue-api`), and compile errors. Imports are checked against `package.json`. Vue-function use is detected when a function name is followed by `(` or `<` in a script that neither imports nor defines it. Each Vue Single-File Component (Vue) (SFC) is compiled with `@vue/compiler-sfc` in the same script-template combination used by Vite's Vue plugin, without type checking or bundling.

- **Trigger.** A problem key becomes active after appearing in two different tasks, following Metis's policy of promoting a procedure only after recurrence [5]. Objectively detected whole-site failures, such as undeclared packages and compile errors, become active after one occurrence. Missing-import and compile-error entries apply to every Vue site. Other entries are used only when the memory model judges their condition to be present in the requirement.

- **Source.** Examples come from earlier websites in the same chain that score at least 0.5, compile, and contain no undeclared import; the highest-scoring eligible site is preferred. The 0.5 threshold is a design choice, not a tuned value. Successful sites show *how* to implement a construct, while failed sites establish *when* the construct may be needed [21].
- **Fragment selection.** The memory model identifies candidate lines, and code verifies that they are verbatim from the source, at most 15 lines long, and contain no `<script>` or `<template>` tag lines. A fragment must include both the import of the most frequently missed function and another line that calls it. If the model fails twice, a rule-based extractor is used. A new version replaces an entry only when it covers strictly more required functions; the earlier version is retained for provenance [35].
- **Slots.** Each task receives at most three entries, with no more than one addressing a missing package.

In this dissertation, *verified* means only that a fragment is copied verbatim from a source file that compiles and comes from a site scoring at least 0.5. It does not guarantee that the fragment is correct for a new task. General triggers also make the pool partly resemble a learned prompt prefix: after activation, many later tasks receive the same fragment. The fixed-prefix ablation (Section 5.5) is designed to distinguish that effect from online learning. Table 4.1 lists the constants of the final configuration.

4.3.3 Memory Hygiene

- **Concrete entries only.** Common-pool entries without a code construct, such as `@click`, `v-if`, or an `import` line, are removed. Rewriting may generalise an applicability *condition* but does not alter the *action*.
- **Deduplication at storage.** A new lesson is compared with similar stored lessons. When the memory model judges that the root cause and fix are the same, the new evidence supports the existing lesson rather than creating another entry [4].
- **Utility and retirement.** For each lesson, the system tracks injections u and recurrences r, where a recurrence is the same problem occurring after injection. Utility is $(u - r + 1)/(u + 2)$ [39, 36]. Entries are ranked by support $\times$ severity $\times$ utility. A lesson injected at least five times with utility below 0.3 is no longer injected. The utility measure is heuristic and its constants were not tuned.

- **No hand-written conclusions.** Reflection instructions specify how to investigate an outcome, rather than prescribing its answer.

Table 4.1: Constants of the final method not stated in the text. Some were revised after development runs (mainly seed 11, Section 5.6.1); all were fixed before the final runs and none was tuned systematically.

Parameter	Value
Lesson retrieval	bge-m3 embeddings; 5 candidates per requirement aspect (Section 4.3), at most 3 functional aspects; similarity ≥ 0.35; only hard matches kept
Lessons injected	at most 3, total length at most $0.5\times$ the requirement
Common pool	6 candidates, at most 3 kept; only entries with a code construct
Deduplication	candidates with similarity ≥ 0.55, then same root cause and fix judged by the memory model
Code-pool trigger	2 tasks; 1 task for whole-site failures
Fragment	verbatim, at most 15 lines, no tag lines
Code-pool slots	at most 3 per task, at most 1 for a missing package

Chapter 5

Results

Each arm is a single run over all 101 tasks. Sections 5.1 to 5.5 use the manifest order; Section 5.6 repeats the comparison in other orders. Accuracy pools all test cases (as in the official script); the learning curves average per-task accuracy, so their end points differ slightly.

5.1 Main Comparison

Table 5.1 reports the primary comparison in the format used by WebGen-Bench [19]. The published WebGen-LM-7B result appears first. Every arm uses the same generator, scaffold, and evaluators. The only treatment is the prompt assembled by the external memory pipeline, including the checklist and any cross-task content.

The final fragment method reaches 35.2%, compared with 28.5% for No Memory. Its paired gain is +7.1 [0.9, 13.1], with 44 wins, 28 losses, and 29 ties, and it reduces zero-score tasks from 30 to 20. Much of this difference comes from sites that run at all: 19 tasks score above zero only with memory, while 9 do so only without it. Among the 62 tasks for which both arms score above zero, the paired gain is +2.6 [−3.2, 8.5]. No Memory varies by only 0.7 points across three orders (Section 5.6). A memory chain varies more because early outcomes change its later memory (Section 5.6.1). The intervals for text memory and the checklist-only control both include zero.

The intermediate whole-file design scores higher in manifest order (40.0%, +11.5 [6.0, 16.9]). The final fragment method is −4.4 [−9.8, 1.0] against it in that order, but the advantage does not persist across task orders (Section 5.6). Table 5.1 assigns an appearance score of zero to tasks without a working site. Restricting the comparison to

Table 5.1: Main results on all 101 tasks in the manifest order (single run per arm). Yes, Partial, No and Start failed are percentages of the 647 test cases; Accuracy = Yes + 0.5× Partial. Appearance is the mean grade over all 101 tasks (1–5; tasks without a working site count as 0). Paired Δ is the mean per-task difference to No Memory with a 95% bootstrap CI; it can differ from the difference of pooled accuracies because tasks have different numbers of test cases. Zero: tasks where every test case fails.

Arm	Yes	Partial	No	Start fail.	Acc.	Appear.	Zero	Paired Δ [95% CI]
WebGen-LM-7B (paper)[a]	24.9	7.1	68.0	0.0	28.4	2.5	–	–
No Memory (ours)	23.8	9.4	66.5	0.3	28.5	2.35	30	–
Text memory	25.2	11.4	62.6	0.8	30.9	2.57	20	+2.6 [−3.6, 8.7]
Checklist only	26.4	10.7	61.4	1.5	31.8	2.47	18	+3.4 [−2.2, 8.7]
Code memory (whole-file)[b]	34.5	11.1	53.2	1.2	40.0	2.83	13	+11.5 [6.0, 16.9]
Code memory (fragments, final)	29.5	11.4	59.0	0.0	35.2	2.67	20	+7.1 [0.9, 13.1]

[a] Reported by Lu et al. [19]; appearance graded by GPT-4o, ours by Qwen2.5-VL-32B, so the two are not directly comparable.

[b] Intermediate design: some code examples were whole files; see Section 5.6.

the 66 tasks for which both No Memory and the final method produced a graded site gives means of 3.23 and 3.24, respectively.

5.2 Learning Curve

Figure 5.1 shows cumulative and rolling accuracy in execution order. During the first 30 tasks, text memory and the final method both outperform No Memory: 39.3% and 35.0%, compared with 26.0%. Over the remaining 71 tasks, text memory falls below No Memory (27.8% versus 29.5%, with 19 zero-score tasks in each arm). The final method remains at a similar level, reaching 35.3% with 14 zero-score tasks; whole-file code memory reaches 40.4%. In the final 35 tasks, the fragment method scores 33.8%, compared with 26.0% for No Memory and 23.2% for text memory.

5.3 Why Text Memory Declines and Code Memory Holds

Table 5.2 records what entered the generator prompt, and Table 5.3 records code problems found by compiling each site with `@vue/compiler-sfc` 3.5.43. A lesson is classified as *lacking a code construct* when it contains none of `@click`, `v-if`, `v-for`, `v-model`, `ref`, an `import`, or inline code. Such lessons, together with a "General notes" block, remain

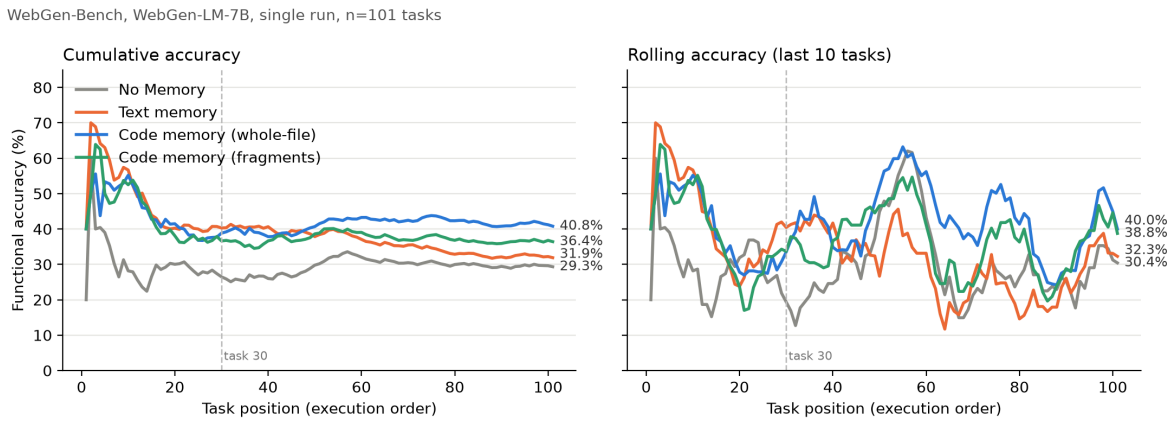


Figure 5.1: Cumulative and rolling-10 accuracy by execution order for No Memory, text memory and both code memories.

common in text memory. They are less frequent in code memory, where a fragment is injected for nearly every task after the pool activates.

Table 5.2: Prompt content by segment (tasks 1–30 / 31–60 / 61–101).

Prompt content	Text	Code (fragments)
Tasks with a lesson without a code construct	28 / 26 / 38	11 / 7 / 19
Tasks with a “General notes” block	13 / 16 / 27	6 / 3 / 4
Tasks with a code-pool fragment	0 / 0 / 0	24 / 30 / 41

Table 5.3: Generated sites with code problems (number of tasks; 1–30 / 31–60 / 61–101, total in brackets).

Problem	No Memory	Text	Code (fragments)
Whole-site break[a]	11 / 5 / 10 (26)	2 / 5 / 12 (19)	4 / 4 / 9 (**17**)
Missing Vue import on a page	7 / 7 / 9 (23)	14 / 10 / 7 (31)	6 / 5 / 10 (21)

[a] An undeclared package import or a Vue compile error. Final method: two sites without code facts are omitted; whole-file code memory had 8 breaks and 22 missing imports.

The problem counts are correlational. The ablations in Section 5.5 are needed to distinguish the contribution of individual components.

5.4 Error Analysis

Whole-site breaks fall from 26 tasks with No Memory to 17 with the final method, and to 8 with whole-file code memory. Missing Vue imports, however, remain as frequent as they are without memory: 4.6% of components in both arms. Because the protocol does not allow repair, this remains a residual generator error.

The final method has 20 zero-score tasks, 11 of which are also zero under No Memory. Nine import an undeclared package: six import `vue-router`, while the others import chart, chess, or editor libraries. Three encounter a Vue compile error involving `v-model` on a file input or prop, three call a Vue function without importing it, and five render but fail every functional test. Only one returns the starter project. `vue-router` is imported without declaration in seven tasks, compared with three under whole-file code memory, even though the relevant fragment is injected in 88 tasks after activation. In manifest order, the fragment prevents this import less reliably than a complete file; under shuffled orders, complete files instead cause scaffold outputs (Section 5.6).

5.5 Ablations: Checklist Only and Fixed Code Prefix

Two complementary ablations use the manifest order: a checklist-only control and a retrospective fixed-code control using the code entries learned by the final method:

- **Checklist only:** the code-memory configuration with memory reads and writes disabled; each task sees only the requirement checklist. Checklist only − No Memory estimates the effect of requirement decomposition.
- **Fixed code prefix:** checklist only, plus the code-pool entries that the final method had learned by the end of its manifest run (Section 4.3.2), injected from the first task with the same selection rule, still without memory. The code is not written by hand; it is the method's own learned content. Because it is taken from the end of a complete run, this control has access to experience that an online learner only gains later; it is a retrospective, oracle-style comparison rather than an online one. Fixed prefix − checklist only estimates the effect of having this code in the prompt, and final method − fixed prefix the effect of accumulating experience and selecting it per task.

If the fixed-code control performs similarly to the final method, the contribution is better described as automatically discovering effective prompt code from the agent's

own failures and successes than as continual accumulation. The comparison does not isolate accumulation alone, because the two conditions also differ in textual lessons and memory maintenance.

Table 5.4 reports the checklist-only ablation over all 101 tasks; the fixed-prefix run follows the interleaved order (expected about 18 Oct).

Table 5.4: Ablation over the 101 tasks in the manifest order (single runs). Accuracy is pooled over checks; appearance averages tasks with a grade.

	Acc. 1–30	Acc. 31–101	Acc. all	Zero-score	Appearance
No Memory	26.0	29.5	28.5	30	3.25
Checklist only	33.1	31.3	31.8	18	3.23
Fixed code prefix			[TODO: running]		
Code memory (whole-file)	39.0	40.4	40.0	13	3.21
Code memory (fragments)	35.0	35.3	35.2	20	3.25

Paired by task, checklist only − No Memory is +3.4 points (95% CI $[-2.2, 8.7]$; 51 wins, 27 losses, 23 ties): requirement decomposition mainly prevents whole-site failures (zero-score tasks 30 → 18) but does not reliably raise accuracy. The final method − checklist only is +3.7 ($[-1.8, 9.1]$; 42/37/22), positive in all three segments of the run but with an interval that includes zero (whole-file: +8.1, $[2.9, 13.2]$); the fixed-prefix comparison will separate the two.

[TODO: Status: fixed-prefix run queued after the interleaved order (about 18 Oct). Fill in its row and the differences fixed prefix − checklist only and final − fixed prefix. Wording depends on the result: if fixed prefix ≈ final, describe the contribution as automatically discovering effective prompt code; if the final method is clearly better, as accumulation and per-task selection.]

5.6 Robustness to Task Order

With memory, the order decides which experience is available; in the manifest order same-domain tasks are adjacent, so a gain could be a curriculum effect. We therefore compare two kinds of order, each run twice from an empty memory:

Domains adjacent. The order of the 20 domain blocks and the order of tasks within each block are shuffled, with seed 11 and seed 47. The manifest order, used for the main results, also belongs to this kind.

Domains mixed. A uniformly random order of all 101 tasks (seed 29), and an interleaved order that takes one task from each domain in turn, so that consecutive tasks always come from different domains.

In the seed-11 and seed-29 orders the two arms run as pairs on each task, alternating which runs first (ABBA). No Memory proved stable: generation is deterministic, and its three complete runs (manifest, seed 11, seed 29) scored 28.5, 27.9 and 28.6% with 30, 29 and 29 zero-score tasks. The seed-47 and interleaved orders were therefore run with Long Memory only and are compared with the per-task mean of these three No Memory runs.

Table 5.5: Task-order experiments: accuracy (%) of No Memory (NM), whole-file code memory (WF) and the final method with fragments (F), paired differences with 95% bootstrap CI over tasks, and zero-score tasks. Single run of each arm per order.

			Accuracy (%)			Paired difference, points [95% CI]			Zero-score
Kind	Order	n	NM	WF	F	WF − NM	F − NM	F − WF	NM / WF / F
Adjacent	Manifest[a, †WF]	101	28.5	**40.0**	35.2	+11.5 [6.0, 16.9]	+7.1 [0.9, 13.1]	−4.4 [−9.7, 1.1]	30 / 13 / 20
Adjacent	Shuffled blocks (seed 11)[†F]	101	27.9	33.1	**35.4**	+4.4 [−1.9, 10.6]	+7.4 [1.6, 13.1]	+3.0 [−3.0, 9.0]	29 / 21 / 16
Adjacent	Shuffled blocks (seed 47)[b]	101	28.3	–	**39.9**	–	+11.7 [5.9, 17.3]	–	27 / – / 9
Mixed	Random (seed 29)	101	28.6	32.6	**33.7**	+4.5 [−2.2, 11.3]	+5.5 [−0.5, 11.6]	+1.0 [−5.0, 7.0]	29 / 21 / 18
Mixed	Interleaved[b]	101	28.3	–	**36.6**	–	+7.9 [1.3, 14.4]	–	27 / – / 12

Bold: best memory arm in the order. †WF / †F: the order on which whole-file code memory / the final method was developed (development data for that arm).
[a] Not run as alternating pairs; the two arms were run at different times.
[b] Long Memory only; NM is the per-task mean of the three complete No Memory runs, hence the same 28.3% in both rows (zero-score: tasks scoring 0 in all three).

Across all five orders the final method stays above No Memory (Figure 5.2): its pooled accuracy is 33.7–39.9%, against 27.9–28.6% for No Memory on the same tasks, and the paired interval excludes zero in four of them (all but seed 29). Read by development data, the table suggests a difference between the two memory designs, although pooled over shared orders they score the same: each is best on the order it was developed on (whole-file on the manifest, +11.5; the final method on seed 11, +7.4), but only the final method keeps a gain of +5.5 to +11.7 on the orders neither was tuned on, where whole-file code memory falls to about +4.5 and the final method is ahead of it by 1.0 to 3.0 points (intervals include zero). Whole-file code memory is ahead of the final method only in the manifest order.

Pooling runs, the per-task mean of the final method's five runs is 36.2%, against 28.3% for the mean of the three No Memory runs: +7.9 [3.1, 12.7], with 68 tasks better and 30 worse. The mean of the three whole-file runs is 35.2%, +6.8 [1.9, 11.7]. Of the

27 tasks that score zero in all three No Memory runs, the final method scores above zero in at least three of its five runs on 21. On the three orders where both designs were run, their means are equal (34.8% and 35.2%; difference −0.1 [−3.6, 3.3]). Excluding each design's development order gives 32.8% for whole-file code memory (+4.4 [−1.0, 9.8]) and 36.7% for the final method (+8.4 [3.2, 13.7]), but these use different orders and are not a head-to-head comparison. Intervals resample tasks only; runs on the same tasks are not independent samples, so these figures summarise the runs rather than add evidence.

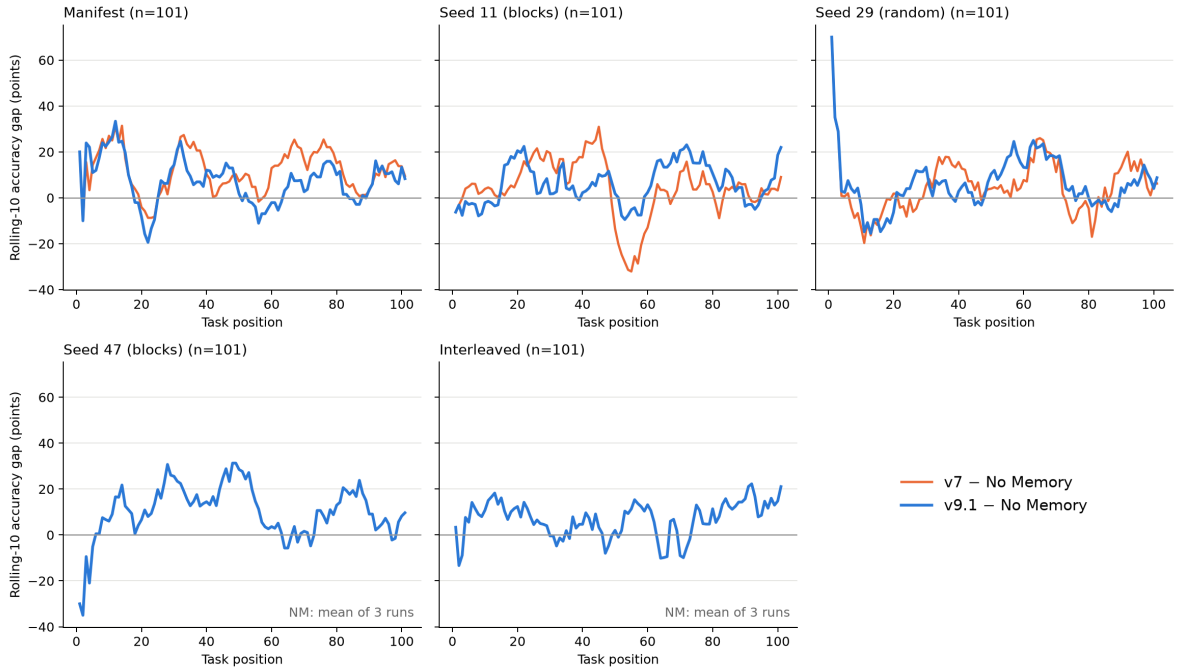


Figure 5.2: Rolling 10-task accuracy gap to No Memory in each order (positive: memory better). Blue: final method; orange: whole-file code memory (manifest, seed 11 and seed 29 only). Seed 47 and interleaved are compared with the per-task mean of three No Memory runs. The orange dip after task 45 in seed 11 coincides with the scaffold outputs (Figure 5.3).

Development and validation. Seed 11 was used to develop the fragment fix (Section 5.6.1) and is therefore development data for the final method. Its manifest, seed-29, seed-47 and interleaved runs started after it was frozen. All orders use the same 101 tasks, on which the method was developed, so this is a within-benchmark study with cross-order validation, not a held-out evaluation (Section 6.3).

In both shuffled orders whole-file code memory leads by about 4.5 points, but neither interval excludes zero (Table 5.5); No Memory stays at 27.9–28.6%, so the change is on

the memory side. The whole-file design was itself developed on the manifest order: each step from text memory to whole-file code memory followed an inspection of failures in that run. Its 40.0% there is therefore likely optimistic, a development-set effect rather than a property of the order, since it falls by about 7 points in both shuffled orders while No Memory does not move. The final method, frozen before all its runs except seed 11, does not depend on the order it was developed on (33.7–39.9% across the five orders, against 32.6–40.0% for whole-file code memory in the three orders where it was run). The random and interleaved orders match the shuffled-blocks orders, so the gain does not require adjacent same-domain tasks, although these runs do not measure how much adjacency contributes.

Both runs lose much of their gain to one failure (Figures 5.2 and 5.3): Long Memory returned the unmodified scaffold on 19 (seed 11) and 16 (seed 29) tasks, all after a complete `App.vue` from another site began to be injected; No Memory never did.

```
CODE FROM EARLIER WEBSITES THAT WORKED:
Missing package: imports a package that
package.json does not declare. [...] Code
that works:
<script setup lang="ts">
import { ref } from 'vue'
import FoodDonationPage from
  './pages/FoodDonationPage.vue'
[... 3 more page imports ...]
const activeTab = ref('donation')
</script>
<template>
  <div class="app-container">
    <header>
      <h1>Food Distribution System</h1>
      [... 30 lines: nav, four pages ...]
</template>
```

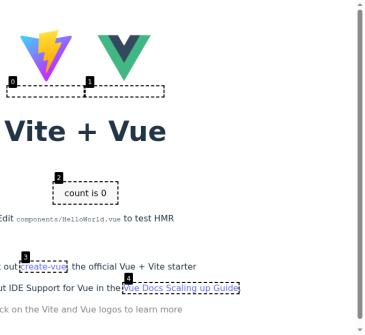


Figure 5.3: Task 000041 (seed 11), whole-file code memory. Left: the injected `App.vue` of an earlier food-donation site. Right: the generator returned the untouched starter project; all five test cases failed.

With the failure fixed (Section 5.6.1), no scaffold occurred in either order (19 → 0 and 16 → 0) and the gain over No Memory rose to 7.4 points in seed 11 (95% CI [1.6, 13.1]) and 5.5 points in seed 29 ([−0.5, 11.6]), with zero-score tasks reduced from 29 to 16 and 18 (Table 5.5 and Figure 5.2). The gain is positive with domains adjacent and mixed (of the two mixed orders, the seed-29 interval includes zero, while the interleaved interval, +7.9 [1.3, 14.4], excludes it), so it does not rely on same-domain neighbours. Against whole-file code memory the final method is ahead by only 3.0 and 1.0 points, within run-to-run variation; the evidence for the fix is the removal of the scaffolds. Missing Vue

imports remain (23 and 27 tasks, against 17 and 19): real pages give more opportunities for this slip.

Over all five orders the gain over No Memory is positive, from +5.5 (seed 29) to +11.7 (seed 47), and its interval excludes zero in four of them. The interleaved order, in which consecutive tasks always come from different domains, is not clearly worse (+7.9 [1.3, 14.4]) than the adjacent orders, so the benefit does not depend on same-domain neighbours.

A positive case. In the manifest order, task 000030 (13th) imported `vue-router`, which the template does not declare, and every test case failed. Static analysis recorded the undeclared import, and the code pool stored a page-switch fragment (a `ref`, navigation buttons and `v-if`, about 15 lines) taken from the 3rd task, a site that had scored 0.92 without a router. Its condition, a requirement with several pages to switch between, held for task 000059 (29th), a study-group site with four sections. Without memory the generator imported `vue-router` and the site did not start (0 of 5 test cases; checklist only also 0); with the fragment it switched sections with `ref` and `v-if` and scored 0.70.[1] The same fragment did not prevent the import in six of the 88 tasks that received it.

5.6.1 Fixing the Exposed Failure

The fix was developed on seed 11 while all other experiments used the frozen whole-file design. It changes only the code pool. Every entry type now shows a fragment rather than a file: on the seven seed-11 tasks where the whole-file design had returned the scaffold, accuracy rose from 3.6% to 45.2%, and no scaffold occurred. An entry is replaced only when its new example covers strictly more of the needed functions, so that the example does not switch back and forth. Whole-site failures create an entry after one occurrence, since waiting for a second meant that rare problems were learned after they had already done their damage, and three slots are allowed instead of two. Compile-error entries apply to every Vue site. Total accuracy on seed 11 is not used to judge these changes: memory chains diverge from the first task, and two runs of the same design can differ by about five points. This is debugging evidence; the full-benchmark evidence for the final method is the set of runs started after it was frozen (Table 5.5).

[1] No Memory code is taken from the seed-11 run; generation is deterministic, and both repeated No Memory runs import `vue-router` on the same 24 tasks.

Chapter 6

Discussion

6.1 The Role of Memory Representation

Across the diagnostic and full runs, this small generator follows short code fragments more reliably than accumulated textual lessons. The text and code systems differ in more than representation, so this is a cross-version observation rather than a controlled causal comparison. It is consistent with Metis's finding that code memory can complement text memory [5], while adding a setting in which the format of the memory may determine whether a 7B generator can use it at all. The form of code also matters: complete files were sometimes copied as finished answers, whereas fragments prevented undeclared imports less reliably.

This pattern differs from reports for stronger models, which transfer text memory more reliably than code [5, 15]. The mechanisms are not identical because Metis executes code whereas this method displays code to the generator. Taken together, the evidence suggests that an appropriate memory representation depends on the capability of the model consuming it.

Part of the observed gain is protection against collapse rather than more capable application logic. In an early phase, short prompts led the model to return the default scaffold in five of eight tasks, whereas longer, specific prompts did not. Both effects can improve system performance, but they should be reported separately.

6.2 What Is Learned and at What Cost

The generator itself does not change. What changes is the code shown to it and the conditions under which it is shown. The resulting behaviour is closer to accumulating

a small set of preventive code examples than to acquiring programming ability. It is also a property of the entire system, including a memory model that is larger than the generator. The memory remains small across 101 tasks; the study does not test what happens over thousands of tasks, when more problem types compete for three slots.

The system also adds computation. A Long Memory task takes approximately 12 minutes, compared with approximately 6 minutes for No Memory, largely because the 72B model constructs checklists, judges applicability, and reflects after each task. If the fixed-prefix ablation recovers most of the gain, a deployed system could use the learned fragments without invoking the 72B model at generation time, at the cost of losing online adaptation. Against this roughly doubled wall time, the final method gains +5.5 to +11.7 points over No Memory across the orders in Table 5.5.

6.3 Limitations and Threats to Validity

- **Single runs.** WebGen-Bench itself reports single runs without error bars. No Memory is repeated under several orders to estimate run-to-run noise, and Long Memory is repeated under further orders using paired comparisons. Within any order, however, Long Memory remains a single run. Two runs of the same Long Memory version can differ by approximately five points, which is more variation than observed for No Memory. The task-level intervals therefore understate full uncertainty. Orders share the same 101 tasks and are not independent samples. [TODO: Status: a second manifest run of the final method starts after seed 47 (results about 18 Oct); report the difference between the two runs.]

- **Gain beyond the checklist and trade-offs between forms.** The final method's paired gain over checklist only is +3.7 [−1.8, 9.1], so an effect beyond requirement decomposition is not established. Whole-file examples also prevent undeclared imports more reliably in the original order, so neither code representation dominates everywhere.

- **Components were added cumulatively.** Each comparison adds one component to the previous arm (No Memory, then checklist only, then checklist with memory), so the effect of memory is measured only in the presence of the checklist (+8.1 [2.9, 13.2] over checklist only for whole-file code memory). Earlier text-memory runs without a checklist also differ in retrieval and condensing, so they cannot separate the two components. Whether memory helps without the checklist, and whether the two interact, is therefore not identified.

- **One generator and one scaffold.** All results concern WebGen-LM-7B, Bolt.diy, and a Vue template. Larger generators may follow textual advice better and gain less from code examples.
- **Code pool as a learned prefix.** The general triggers make the code pool partly resemble a fixed prompt prefix. The fixed-prefix ablation (Section 5.5) is intended to measure how much of the effect this accounts for.
- **Rare problems are learned late.** A safeguard can appear only after the relevant problem has occurred. Memory may therefore help frequent problems more than rare ones, and several problem types can compete for three slots (Section 5.6.1).
- **Evaluators.** Appearance uses Qwen2.5-VL-32B rather than GPT-4o and is therefore compared only across the present arms. GUI testers can exhaust their interaction budget or misjudge pages [37]. Raw transcripts are retained, but the judgements were not independently reviewed by a human.
- **Iterative development on the same benchmark.** This is the most serious limitation. Every version was designed after failures were observed on the same 101 tasks used for evaluation. The effect is visible: whole-file code memory, developed on the manifest order, scores 40.0% there but 33.1% and 32.6% in the two shuffled orders, while No Memory stays at 27.9–28.6%. Empty memories and freezing the final version before all runs except seed 11 reduce this risk but do not remove it. A held-out evaluation is required. The split into the first 30 and the remaining 71 tasks is a learning-curve analysis, not a held-out test, because development looked at all 101 tasks.
- **Memory model stronger than generator.** The 72B memory model exceeds the 7B generator in size, so some gains may reflect the larger model's organisation of the task. The checklist only restates the requirement, every fragment is copied verbatim from an earlier site generated by the 7B model, and the memory model never sees the current task's evaluation. The results therefore concern what a small generator gains when a stronger model organises its prior experience, rather than self-improvement in isolation. [TODO: Status: fixed-prefix run queued after seed 47. State how much of the gain remains without accumulated memory and without the 72B model at generation time.]

6.4 Ethical Considerations and Impact

Data and privacy. The benchmark requirements are synthetic and contain no personal data. All models run on university hardware, and no task data are sent to external services.

Environmental cost. A full run occupies two H200 GPUs for about one day (Section 3.5). To limit unnecessary computation, changes were first checked on three to five tasks and with static code analysis before full evaluation.

Self-improving agents and society. A memory system that turns outcomes into later behaviour can preserve mistakes as readily as useful practices. This design retains provenance for every memory entry so that injected material can be traced, but systems that store and reuse code in deployment would still require security review. Small local models can reduce website-development costs for smaller organisations, while potentially displacing routine front-end work. Their output remains subject to human review.

Use of AI tools. Claude and ChatGPT were used to assist with programming the experimental pipeline and analysis scripts, organising experiment logs, and giving suggestions in critical reviews of drafts; Gemini and Microsoft Copilot were also used for review comments. All text was written by the author, who decided which suggestions to adopt. Research decisions and the verification of all results are the author's responsibility; every number was computed by the scripts in the public repository and checked by the author. [TODO: Author: this statement holds only after the final text has been rewritten in your own words; check it against what was actually done before submission.]

Research integrity. We report negative results (text memory's decline, the scaffold failure of whole-file examples) and invalidated our own earlier results when a delivery bug was found.

6.5 Project Management

The project was organised in versions. The cluster ran one frozen version at a time; new ideas were tested locally on small samples and with static code checks, and a version was replaced only when the new one was shown to be better. Decisions and their evidence

were recorded continuously in a decision log, which made it possible to reconstruct why each component exists.

Chapter 7

Conclusion

7.1 Answers to the Research Questions

RQ1 The answer is a qualified yes for WebGen-LM-7B with Bolt.diy on WebGen-Bench. The final code-memory method raises accuracy from 28.5% to 35.2% (+7.1, with a confidence interval excluding zero) and reduces zero-score tasks from 30 to 20. It also gains 7.4 and 5.5 points in two shuffled orders. Most of the gain comes from sites that render successfully. The gain is positive in all five task orders (+5.5 to +11.7 points), with an interval excluding zero in four. Averaged over its five runs, the gain is +7.9 points [3.1, 12.7].

RQ2 In these runs, the small generator followed short code fragments from its earlier websites more readily than textual lessons. Text memory fell below No Memory late in the sequence, although this is a cross-version rather than controlled comparison. The checklist-only control adds little (+3.4, with an interval including zero). Fragments also need to remain partial: complete files can be copied as finished answers. [TODO: Status: add the fixed-prefix result (queued after seed 47).]

RQ3 Code memory maintains an advantage over the sequence (35.3% versus 29.5% on tasks 31–101), whereas text memory declines. This pattern is consistent with learning from experience but does not prove it. Whole-file examples cause scaffold outputs when order changes. Verified fragments remove those scaffolds, and their gain is positive in all five orders, with domains both adjacent and mixed; the interval includes zero only for seed 29. Whole-file code memory scores higher only in the original order (40.0%). Pooled over the orders run with both designs, fragments and whole files score the same;

the advantage of fragments is that they avoid scaffold outputs and do not depend on the order they were developed on, not that they score higher.

7.2 Future Work

Two experiments would address the most important remaining uncertainties.

- **Held-out evaluation.** Because the method was developed while observing results on all 101 tasks, a clean evaluation requires tasks that were not used in development. Suitable options include a held-out domain or a second website-generation benchmark, with the method frozen in advance.
- **Controlled comparison of memory forms.** Retrieval, trigger conditions, and token budget should be fixed while varying only the representation: textual rule, pseudocode, short fragment, complete file, plus random-code and length-matched-text controls.
- **A second generator.** Repeating the comparison with a stronger generator, such as WebGen-LM-32B, under the same pipeline would show whether code fragments remain more useful than textual lessons when the generator follows text better.

Other priorities are repeated runs for each order, which would separate order effects from run-to-run variation, and testing larger generators from the same family with a text-memory arm to determine whether the preference for code depends on model size.

Bibliography

[1] Shuai Bai et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025. URL: https://arxiv.org/abs/2502.13923.

[2] Jianlv Chen et al. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024. URL: https://arxiv.org/abs/2402.03216.

[3] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023. URL: https://arxiv.org/abs/2304.05128.

[4] Prateek Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2504.19413.

[5] Zijie Dai et al. Metis: Bridging text and code memory for self-evolving agents. *arXiv preprint arXiv:2606.24151*, 2026. URL: https://arxiv.org/abs/2606.24151.

[6] Xiang Deng et al. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023. URL: https://arxiv.org/abs/2306.06070.

[7] Shehzaad Dhuliawala et al. Chain-of-verification reduces hallucination in large language models. *arXiv preprint arXiv:2309.11495*, 2023. URL: https://arxiv.org/abs/2309.11495.

[8] Yufeng Du, Minyang Tian, et al. Context length alone hurts LLM performance despite perfect retrieval. In *Findings of EMNLP*, 2025. arXiv:2510.05381. URL: https://arxiv.org/abs/2510.05381.

[9] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. URL: https://doi.org/10.1201/9780429246593.

[10] Keno Harada et al. Curse of instructions: Large language models cannot follow multiple instructions at once. *arXiv preprint*, 2025. URL: https://openreview.net/forum?id=R6q67CDBCH.

[11] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of ACL*, 2024. URL: https://arxiv.org/abs/2401.13919.

[12] Binyuan Hui et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL: https://arxiv.org/abs/2409.12186.

[13] Daniel Jaroslawicz et al. How many instructions can LLMs follow at once? *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2507.11538.

[14] Carlos E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL: https://arxiv.org/abs/2310.06770.

[15] Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents. *arXiv preprint arXiv:2604.14004*, 2026. URL: https://arxiv.org/abs/2604.14004.

[16] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL: https://arxiv.org/abs/2309.06180.

[17] Yuetai Li et al. Small models struggle to learn from strong reasoners. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2502.12143.

[18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. URL: https://arxiv.org/abs/2307.03172.

[19] Zimu Lu et al. WebGen-Bench: Evaluating LLMs on generating interactive and functional websites from scratch. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2505.03733.

[20] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL: https://arxiv.org/abs/2303.17651.

[21] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *Proceedings of ICLR*, 2026. arXiv:2509.25140. URL: https://arxiv.org/abs/2509.25140.

[22] Charles Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL: https://arxiv.org/abs/2310.08560.

[23] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of UIST*, 2023. URL: https://arxiv.org/abs/2304.03442.

[24] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of ICML*, 2023. URL: https://arxiv.org/abs/2302.00093.

[25] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL: https://arxiv.org/abs/2303.11366.

[26] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. AgentCL: Toward rigorous evaluation of continual learning in language agents. *arXiv preprint arXiv:2606.02461*, 2026. URL: https://arxiv.org/abs/2606.02461.

[27] Chenglei Si et al. Design2Code: Benchmarking multimodal code generation for automated front-end engineering. *arXiv preprint arXiv:2403.03163*, 2024. URL: https://arxiv.org/abs/2403.03163.

[28] Daegyu Sung, Yukyeong Lee, Geon Park, Yumin Choi, and Sung Ju Hwang. Super library agent: Joint generation and maintenance of multiple applications beyond the single codebase. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2608.29310. URL: https://arxiv.org/abs/2608.29310.

[29] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of EACL*, 2026. URL: https://arxiv.org/abs/2504.07952.

[30] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL: https://arxiv.org/abs/2404.13208.

[31] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL: https://arxiv.org/abs/2305.16291.

[32] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of ICML*, 2025. URL: https://arxiv.org/abs/2409.07429.

[33] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, et al. Evomemory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025. URL: https://arxiv.org/abs/2511.20857.

[34] Andreas Zeller. *Why Programs Fail: A Guide to Systematic Debugging*. Morgan Kaufmann, 2 edition, 2009. URL: https://doi.org/10.1016/B978-0-12-374515-6.X0000-7.

[35] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *Proceedings of ICLR*, 2026. arXiv:2510.04618. URL: https://arxiv.org/abs/2510.04618.

[36] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. URL: https://arxiv.org/abs/2308.10144.

[37] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023. URL: https://arxiv.org/abs/2306.05685.

[38] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *Proceedings of ICLR*, 2024. URL: https://arxiv.org/abs/2306.07863.

[39] Huichi Zhou et al. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint*, 2025. URL: https://arxiv.org/abs/2508.16153.

[40] Shuyan Zhou et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL: https://arxiv.org/abs/2307.13854.